

Python Programming with Artificial Intelligence

Programming with LLM

Huangxun Chen @IoT&AI Thrust



- We have learned how to use Python to build and train AI models, ranging from conventional machine learning algorithms to deep learning models such as CNNs, RNNs, and Transformers.
- Training large language models from scratch is challenging, but integrating pretrained LLMs into applications is both practical and powerful.
- In this lecture, we will learn several practical techniques and frameworks for building LLM-powered applications and integrating LLM capabilities into Python programs.

Advanced Prompting & Reasoning

- zero-shot, few-shot, delimiters, describe the request in numbered Steps
- Chain-of-Thought (CoT) Prompting, Output Formatting
- Role in Conversation
- Self-Consistency (SC)

AI Agent

- Agent Development Frameworks: LangChain
- External Memory: RAG
- Text Embeddings: SentenceTransformers
- Vector Database: ChromaDB
- Agent & Tool Use



Advanced Prompting & Reasoning

A Practical Scenario: Sanitizing Customer Chat Conversations

- You are a developer at a company that handles thousands of customer support chats on a daily basis.
- You are supposed to make these text conversations more accessible for further processing by the customer support department:
 - Remove personally identifiable information.
 - Clean the date-time information to only show the date.
 - Remove swear words.
 - Mark the conversations as “positive” or “negative.”
 - Format the output as JSON.

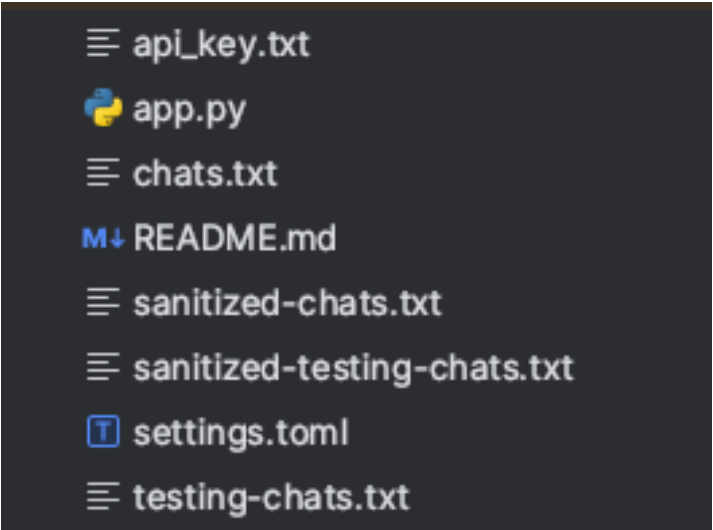
[support_tom] 2023-07-24T10:02:23+00:00 : What can I help you with?
[johndoe] 2023-07-24T10:03:15+00:00 : I CAN'T CONNECT TO MY BLASTED ACCOUNT
[support_tom] 2023-07-24T10:03:30+00:00 : Are you sure it's not your caps lock?
[johndoe] 2023-07-24T10:04:03+00:00 : Blast! You're right!



[Agent] 2023-07-24 : What can I help you with?
[Customer] 2023-07-24 : I CAN'T CONNECT TO MY 🙄 ACCOUNT
[Agent] 2023-07-24 : Are you sure it's not your caps lock?
[Customer] 2023-07-24 : 🙄! You're right!

A Practical Scenario: Sanitizing Customer Chat Conversations

- Download the codebase from Canvas.
- create and activate a new virtual environment.
- use pip to install the required dependencies



A screenshot of a file explorer window with a dark background. It lists several files and folders: 'api_key.txt' with a folder icon, 'app.py' with a Python logo icon, 'chats.txt' with a folder icon, 'README.md' with a document icon and a blue 'M' icon, 'sanitized-chats.txt' with a folder icon, 'sanitized-testing-chats.txt' with a folder icon, 'settings.toml' with a document icon and a blue 'T' icon, and 'testing-chats.txt' with a folder icon.

settings.toml: hosts the prompts. Keeping your prompts in a dedicated settings file can help to put them under version control.

App.py: read the prompts from settings.toml, assemble them meaningfully, and send an API requests to LLM.

```
(venv) $ python app.py chats.txt
```

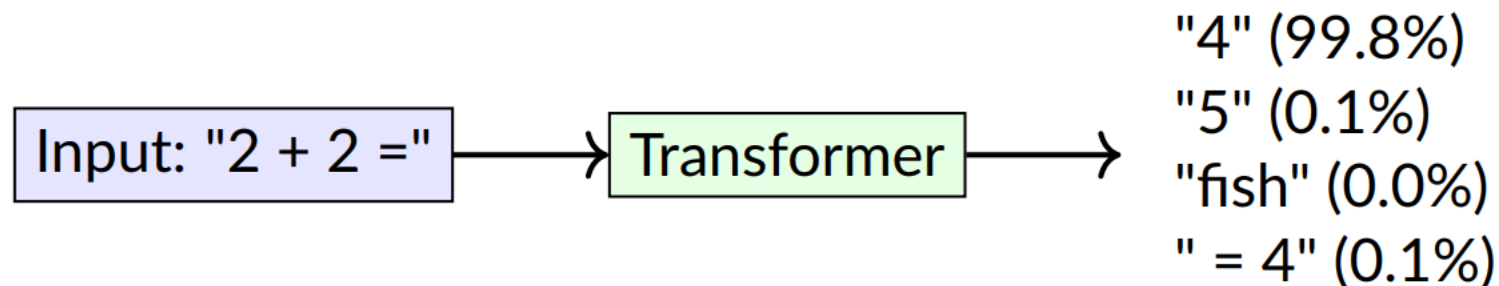
Review: The Statistical Nature of LLMs

LLMs are **Next Token Predictors** - they predict the most probable next token based on previous tokens

- They model $P(w_t | w_{t-k}, \dots, w_{t-1})$: the probability of the next word given the previous context.

LLMs are sophisticated autocomplete systems predicting next tokens from patterns, not true problem solvers.

- They don't understand like humans; they recognize statistical patterns from training data.



Freeze Responses of LLM

Problem: When you are planning to integrate an LLM into a workflow, then you may generally want deterministic responses. The same input should give you the same output. Otherwise, it gets hard to provide a consistent service or debug your program if something goes wrong.

Solution: Set the **temperature** to 0, you can get **mostly deterministic** results

- Higher temperature settings will introduce more randomness into the results by allowing the LLM to pick tokens with lower probabilities.

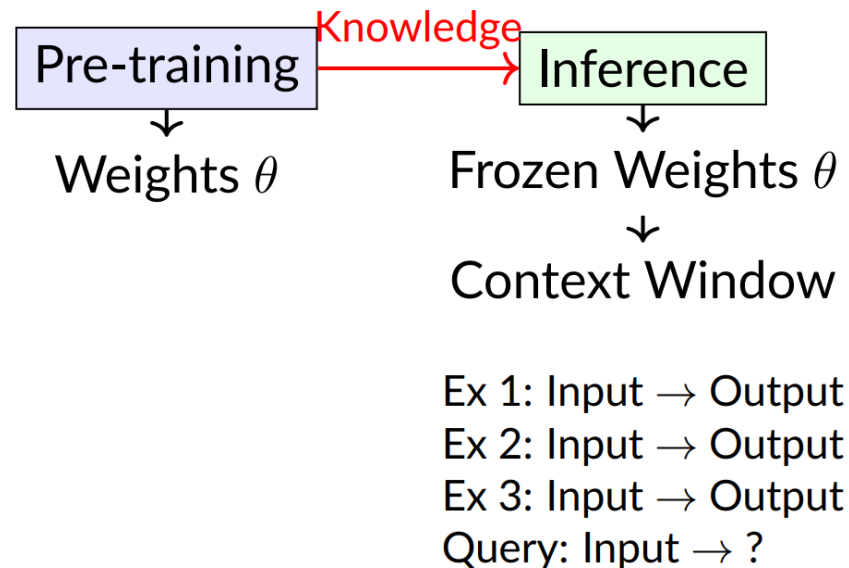
```
[general]  
model = "gpt-3.5-turbo"  
temperature = 0
```

In-Context Learning (ICL)/Prompt Engineering

ICL = Learn tasks from examples in prompt without gradient updates

- **Training:** Weights θ change (pre-training)
- **ICL:** Weights θ frozen (inference-time)

ICL turns LLMs into **few-shot learners** without retraining.



■ A Practical Scenario: Sanitizing Customer Chat Conversations

Zero-Shot Prompting: The model attempts the task with no examples.

```
instruction_prompt = """  
Remove personally identifiable information, only show the date, and replace all  
swear words with "🙄"  
"""
```

■ Tips: Use few-shot examples to establish patterns

```
instruction_prompt = ""
```

```
Remove personally identifiable information, only show the date, and replace all swear words with  
"🙄"
```

Example Input:

```
[support_tom] 2023-07-24T10:02:23+00:00 : What can I help you with?
```

```
[johndoe] 2023-07-24T10:03:15+00:00 : I CAN'T CONNECT TO MY BLASTED ACCOUNT
```

```
[support_tom] 2023-07-24T10:03:30+00:00 : Are you sure it's not your caps lock?
```

```
[johndoe] 2023-07-24T10:04:03+00:00 : Blast! You're right!
```

Example Output:

```
[Agent] 2023-07-24 : What can I help you with?
```

```
[Customer] 2023-07-24 : I CAN'T CONNECT TO MY 🙄 ACCOUNT
```

```
[Agent] 2023-07-24 : Are you sure it's not your caps lock?
```

```
[Customer] 2023-07-24 : 🙄! You're right!
```

```
""
```

■ Tips: Choose Representative Examples

```
instruction_prompt = ""
```

```
...
```

Example Input:

```
...
```

[support_amy] 2023-06-15T14:45:35+00:00 : Hello! How can I assist you today?

[greg_stone] 2023-06-15T14:46:20+00:00 : I can't seem to find the download link for my purchased software.

[support_amy] 2023-06-15T14:47:01+00:00 : **No problem, Greg**. Let me find that for you. Can you please provide your order number?

[greg_stone] 2023-06-15T14:47:38+00:00 : **It's 1245789**. Thanks for helping me out!

Example Output:

```
...
```

[Agent] 2023-06-15 : Hello! How can I assist you today?

[Customer] 2023-06-15 : I can't seem to find the download link for my purchased software.

[Agent] 2023-06-15 : **No problem, ******. Let me find that for you. Can you please provide your order number?

[Customer] 2023-06-15 : **It's ******. Thanks for helping me out!

```
""
```

Tips: Use Delimiters

- Everything you write arrives to an LLM as a single prompt, a long sequence of tokens.
- Use **Delimiters** to clearly mark sections of your prompt

```
def _assemble_chat_messages(content: str) -> list[dict]:
    """Combine all messages into a well-formatted list of dicts."""
    messages = [
        {"role": "system", "content": SETTINGS["prompts"]["role_prompt"]},
        {"role": "user", "content": SETTINGS["prompts"]["negative_example"]},
        {
            "role": "system",
            "content": SETTINGS["prompts"]["negative_reasoning"],
        },
        {
            "role": "assistant",
            "content": SETTINGS["prompts"]["negative_output"],
        },
        {"role": "user", "content": SETTINGS["prompts"]["positive_example"]},
        {
            "role": "system",
            "content": SETTINGS["prompts"]["positive_reasoning"],
        },
        {
            "role": "assistant",
            "content": SETTINGS["prompts"]["positive_output"],
        },
        {"role": "user", "content": f">>>>\n{content}\n<<<<"},
        {"role": "user", "content": SETTINGS["prompts"]["instruction_prompt"]},
    ]
    return messages
```

Tips: Use Delimiters

- Delimiters help to separate the content and examples from the task description.
- A delimiter can be any sequence of characters that usually wouldn't appear together

For example:

- >>>>>
- =====
- #####
- START CONTENT>>>>> content <<<<<END CONTENT
- ===== START content END =====
- ##### START EXAMPLES examples ##### END EXAMPLES

■ Tips: Use Delimiters

- Delimiters can also make it possible to refer to specific parts of your prompt at a later point in the prompt.

```
instruction_prompt = """Remove personally identifiable information from >>>>CONTENT<<<<,
only show the date, and replace all swear words with "🙄" """
```

```
#### START EXAMPLES
```

```
----- Example Inputs -----
```

```
...
```

```
----- Example Outputs -----
```

```
...
```

```
#### END EXAMPLES
```

```
"""
```

Test current prompt across different data

- Mixing training, validation, and testing data is a bad practice in ML/AI.
- We wonder how well the prompt generalizes to different input to avoid overfitting.
- Try change model from “gpt-3.5-turbo” to “gpt-4”.

\$ python app.py testing-chats.txt

■ Tips: Describe the Request in Numbered Steps

Break up your task instructions into a numbered sequence of small steps
→ make your instructions more specific

```
instruction_prompt = ""
```

Sanitize the text provided in >>>CONTENT<<< in multiple steps:

1. Replace personally identifiable information (customer names, agent names, email addresses, order numbers) with `\*\*\*\*`
2. Replace names in [] with "Agent" and "Client", respectively
3. Replace the date-time information to only show the date in the format YYYY-mm-dd
4. Replace all soft and hard swear words with the following emoji: "🤦"

START EXAMPLES

...

####

Task: Classify the Sentiment of Chat Conversations

Start by saving both sanitized conversation files into new files that will constitute the new inputs for your sentiment classification task:

```
(venv) $ python app.py chats.txt > sanitized-chats.txt
```

Task: Classify the Sentiment of Chat Conversations

```
instruction_prompt = ""
```

```
Classify the sentiment of each conversation in >>>>CONTENT<<<< with "🔥" for negative and "✅" for positive:
```

```
#### START EXAMPLES
```

```
----- Example Inputs -----
```

```
[Agent] 2023-07-24 : What can I help you with?
```

```
[Customer] 2023-07-24 : I CAN'T CONNECT TO MY 😞 ACCOUNT
```

```
[Agent] 2023-07-24 : Are you sure it's not your caps lock?
```

```
[Customer] 2023-07-24 : 😞 ! You're right!
```

```
...
```

```
----- Example Outputs -----
```

```
🔥
```

```
[Agent] 2023-07-24 : What can I help you with?
```

```
[Customer] 2023-07-24 : I CAN'T CONNECT TO MY 😞 ACCOUNT
```

```
[Agent] 2023-07-24 : Are you sure it's not your caps lock?
```

```
[Customer] 2023-07-24 : 😞 ! You're right!
```

```
...
```

```
#### END EXAMPLES
```

```
""
```

The Limit of Direct Prompting

Task: “A juggler has 16 balls. Half are golf balls. Half of the golf balls are blue. How many blue golf balls?”

Direct Answer: "8" (Incorrect)

Why it fails:

- Shallow association: "Half of 16 is 8"
- Misses the second "Half"
- LLMs are pattern matchers, not reasoners

When **problems require intermediate reasoning steps**, we need techniques that force "showing work".

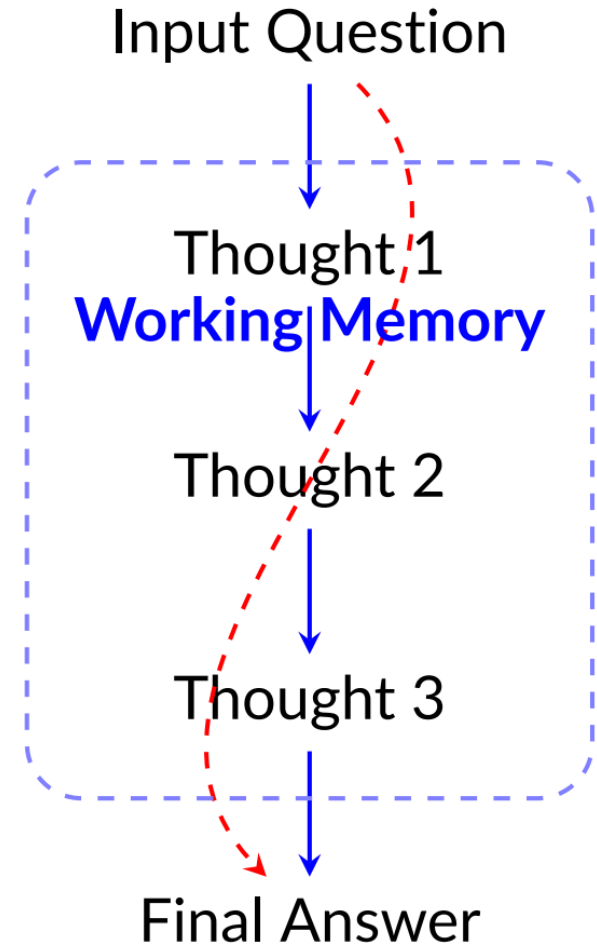
Chain-of-Thought (CoT) Prompting

CoT transforms LLM from pattern matcher to reasoning engine by providing working memory.

Core Idea: LLMs need "compute time" for hard problems

How it works:

- Allocate more tokens to the problem
- Create a "scratchpad" for intermediate results
- Mimic human step-by-step reasoning

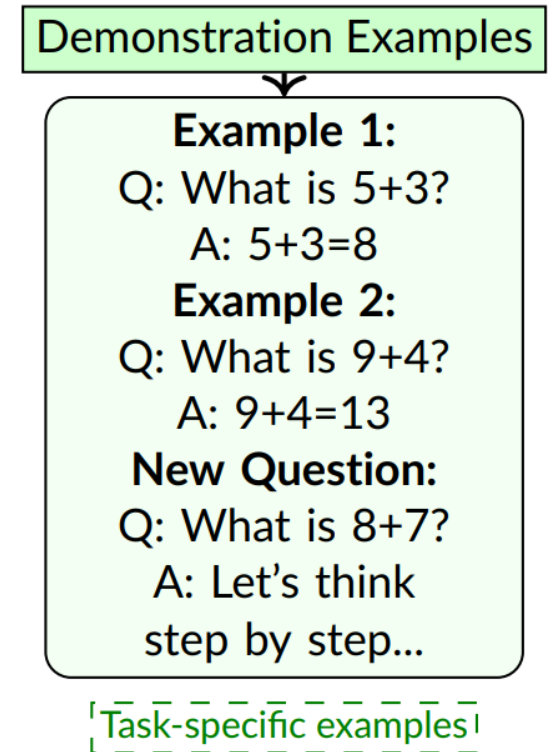
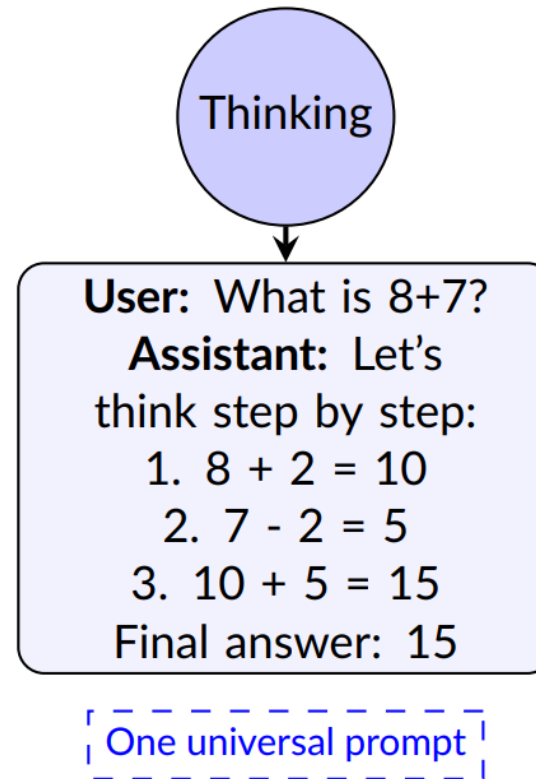


Zero-Shot CoT

- prompt: Let's think step by step
- general-purpose
- easy implementation
- unstructured reasoning

Manual CoT

- Example-based reasoning
- domain-specific
- higher performance
- require examples



■ Tips: Add CoT

Remove the examples and replace the instructions describing the reasoning on how you would classify the conversations in more detail:

```
instruction_prompt = ""
```

Classify the sentiment of each conversation in >>>>CONTENT<<<< with "🔥" for negative and "✅" for positive.

Follow these steps when classifying the conversations:

1. Does the customer use swear words or 🙄?
2. Does the customer seem aggravated or angry?

If you answer "Yes" to one of the above questions, then classify the conversation as negative with "🔥". Otherwise classify the conversation as positive with "✅".

Let's think step by step

```
""
```

■ Tips: Add CoT

You'll get back a reference to the conversations, with the reasoning spelled out step by step to reach the final conclusion:

1. Conversation 1:

- Does the customer use swear words or 🤔? Yes
- Does the customer seem aggravated or angry? Yes
- Sentiment: 🔥

2. Conversation 2:

- Does the customer use swear words or 🤔? No
- Does the customer seem aggravated or angry? No
- Sentiment: ✅

...

■ Tips: Add CoT

You can now use this information to enhanced each of the examples with a short chain of thought that you generated in the previous call.

----- Example Inputs -----

[Agent] 2023-07-24 : What can I help you with?

[Customer] 2023-07-24 : I CAN'T CONNECT TO MY 🙄 ACCOUNT

[Agent] 2023-07-24 : Are you sure it's not your caps lock?

[Customer] 2023-07-24 : 🙄! You're right!

- Does the customer use swear words or 🙄? Yes
- Does the customer seem aggravated or angry? Yes
- Sentiment: 🔥

[Agent] 2023-06-15 : Hello! How can I assist you today?

[Customer] 2023-06-15 : I can't seem to find the download link for my purchased software.

[Agent] 2023-06-15 : No problem, ****. Let me find that for you. Can you please provide your order number?

[Customer] 2023-06-15 : It's ****. Thanks for helping me out!

- Does the customer use swear words or 🙄? No
- Does the customer seem aggravated or angry? No
- Sentiment: ✅

Task: Format the output as JSON

```
instruction_prompt = ""
```

Classify the sentiment of each conversation in

```
>>>>CONTENT<<<< as "negative" and "positive".
```

Return the output as valid JSON.

```
#### START EXAMPLES
```

```
----- Example Inputs -----
```

[Agent] 2023-07-24 : What can I help you with?

[Customer] 2023-07-24 : I CAN'T CONNECT TO MY  ACCOUNT

[Agent] 2023-07-24 : Are you sure it's not your caps lock?

[Customer] 2023-07-24 :  ! You're right!

- Does the customer use swear words or  ? Yes
- Does the customer seem aggravated or angry? Yes
- Sentiment: "negative"

[Agent] 2023-06-15 : Hello! How can I assist you today?

[Customer] 2023-06-15 : I can't seem to find the download link for my purchased software.

[Agent] 2023-06-15 : No problem, ****. Let me find that for you. Can you please provide your order number?

[Customer] 2023-06-15 : It's ****. Thanks for helping me out!

- Does the customer use swear words or  ? No
- Does the customer seem aggravated or angry? No
- Sentiment: "positive"

```
----- Example Output -----
```

```
{  
  "negative": [  
    {  
      "date": "2023-07-24",  
      "conversation": [  
        "A: What can I help you with?",  
        "C: I CAN'T CONNECT TO MY  ACCOUNT",  
        "A: Are you sure it's not your caps lock?",  
        "C:  ! You're right!"  
      ]  
    },  
    "positive": [  
      {  
        "date": "2023-06-15",  
        "conversation": [  
          "A: Hello! How can I assist you today?",  
          "C: I can't seem to find the download link for my  
purchased software.",  
          "A: No problem, ****. Let me find that for you. Can you  
please provide your order number?",  
          "C: It's ****. Thanks for helping me out!"  
        ]  
      }  
    ]  
  }  
}#### END EXAMPLES""
```

Test current prompt across different data

- Mixing training, validation, and testing data is a bad practice in ML/AI.
- We wonder how well the prompt generalizes to different input to avoid overfitting.

\$ python app.py sanitized-testing-chats.txt

Improve Output With the Power of Conversation

Instead of a long prompt, we can provide additional context to the model.

A prompt can be split into several messages.

Each message has its **role** and **prompt text**.

- "system" role gives context for the conversation and helps to set the overall tone.
- "user" role represents the input that a user of your application might provide.
- "assistant" role represents the output that the model would reply with.

■ Tips: Add a Role Prompt

Role prompting usually refers to adding system messages, which represent information that helps to set the context for upcoming completions that the model will produce.

```
instruction_prompt = """
Classify the sentiment of each conversation in >>>>CONTENT<<<< with "🔥" for negative and "✅" for
positive:

#### START EXAMPLES
...
#### END EXAMPLES
"""

role_prompt = """You are a thoroughly trained machine learning model that is an expert at sentiment
classification. You diligently complete tasks as instructed. You never make up any information that isn't there."""
```

Tips: Separate Example, CoT Reasoning and Outputs

```
instruction_prompt = """
Classify the sentiment of each conversation in >>>>CONTENT<<<< as "negative" and "positive".
Return the output as valid JSON.
"""

role_prompt = """You are a thoroughly trained machine learning model that is an expert at sentiment
classification. You diligently complete tasks as instructed. You never make up any information that isn't there."""
positive_example = """
[Agent] 2023-06-15 : Hello! How can I assist you today?
[Customer] 2023-06-15 : I can't seem to find the download link for my purchased software.
[Agent] 2023-06-15 : No problem, ****. Let me find that for you. Can you please provide your order number?
[Customer] 2023-06-15 : It's ****. Thanks for helping me out!
"""

positive_reasoning = """
- Does the customer use swear words or 🙄? No
- Does the customer seem aggravated or angry? No
- Sentiment: "positive"
"""

positive_output = """
...
"""
```

Removed the delimiters that we previously
used for labeling the example sections.

■ Tips: Separate Example, CoT Reasoning and Outputs

Each message has its **role** and **prompt text**.

```
# ...

def _assemble_chat_messages(content: str) -> list[dict]:
    """Combine all messages into a well-formatted list of dicts."""
    messages = [
        {"role": "system", "content": settings.role_prompt},
        {"role": "user", "content": settings.negative_example},
        {"role": "system", "content": settings.negative_reasoning},
        {"role": "assistant", "content": settings.negative_output},
        {"role": "user", "content": settings.positive_example},
        {"role": "system", "content": settings.positive_reasoning},
        {"role": "assistant", "content": settings.positive_output},
        {"role": "user", "content": f">>>>\n{content}\n<<<<"},
        {"role": "user", "content": settings.instruction_prompt},
    ]
    return messages
```

Self-Consistency (SC)

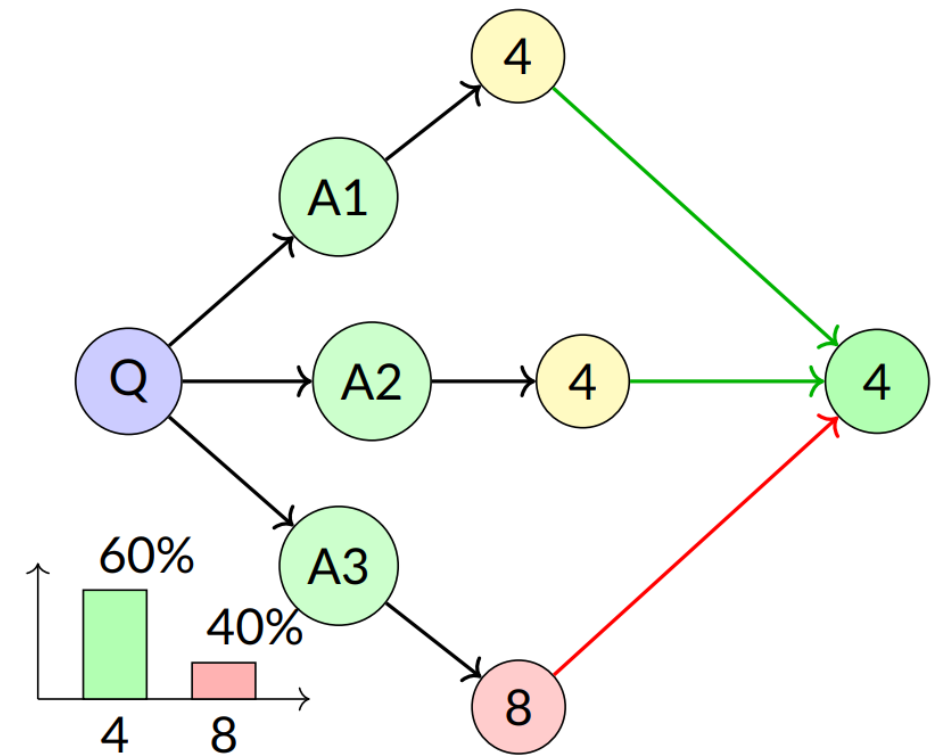
Self-Consistency transforms unreliable reasoning into **statistically robust** solutions.

Problem: Single reasoning path is fragile

Solution: Sample multiple paths and vote

Algorithm:

- Sample k outputs (Temperature > 0)
- Parse final answers
- Return majority vote



In-Class Practice (Submit on Canvas)

Based on the project file, edit settings.toml and app.py to apply the Self-Consistency (SC) technique ($k=3$) for chat sentiment classification.

Test on sanitized-chats.txt and sanitized-testing-chats.txt.



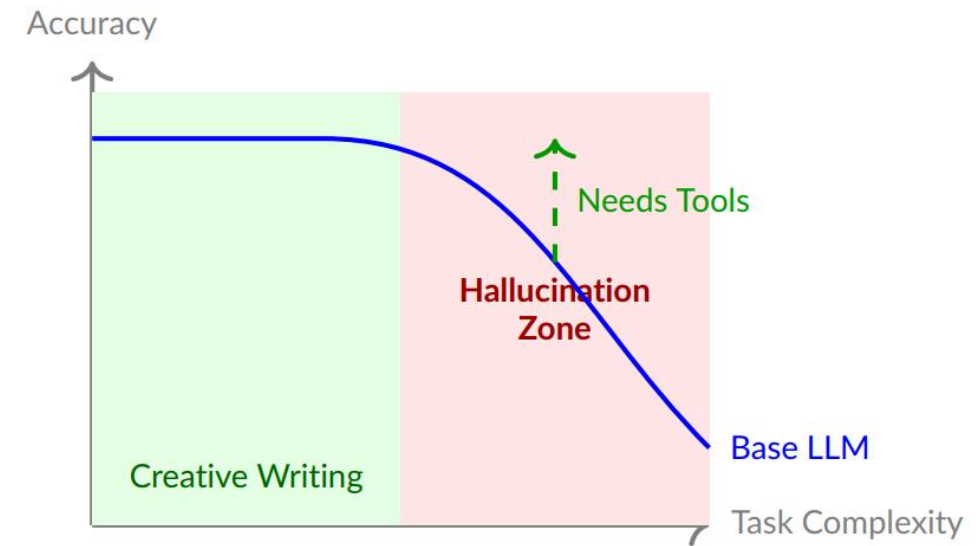
AI Agent

■ The Problem: LLMs are "Frozen" and "Fuzzy"

What base LLMs **cannot** do:

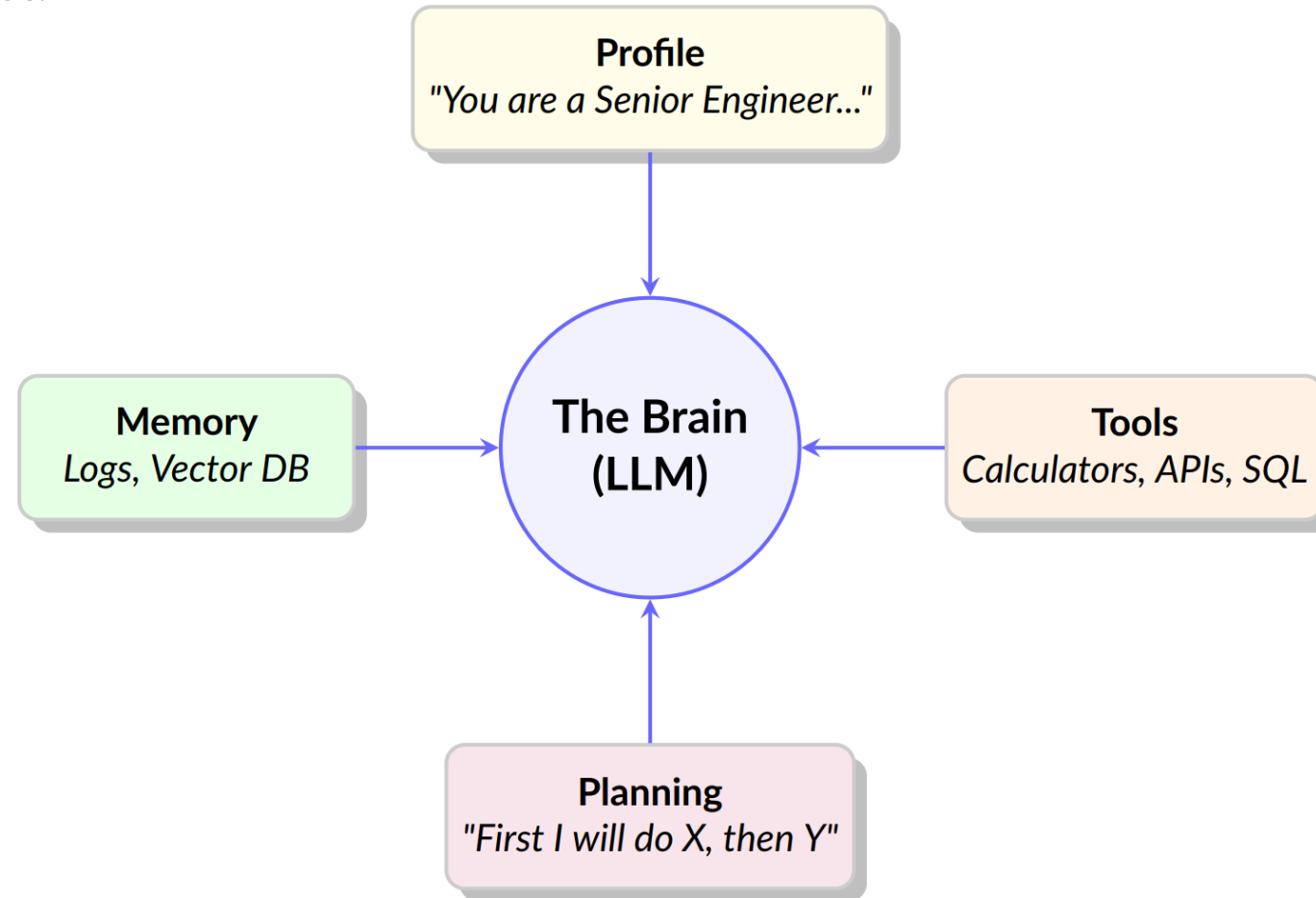
- **No Current Knowledge:** Training data cutoffs (e.g., "What is Apple's stock price right now?")
- **No Action:** They output text, they cannot click buttons, send emails, or run code.
- **Bad at Exact Math:** They are probabilistic token predictors, not calculators.

Solution: We need to give the LLM "Hands" (Tools) and "Eyes" (Search).



What is an AI Agent?

An Agent is an LLM system equipped with an environment to perceive and act.



Why Use Frameworks?

- Simplify complex agent implementations
- Provide pre-built tools and integrations
- Handle edge cases and error handling
- Enable rapid prototyping and iteration

Comparison

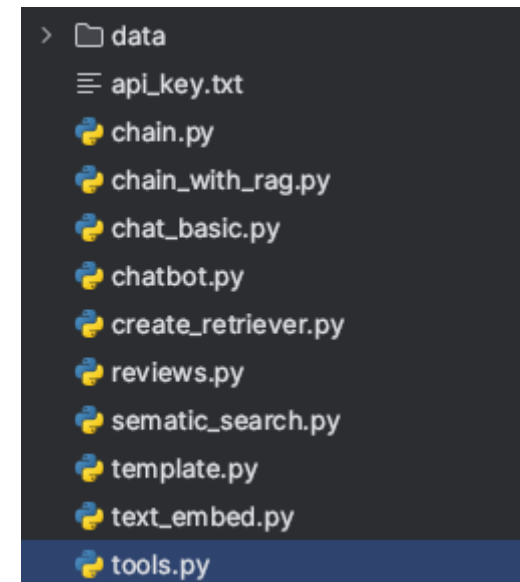
Framework	Best For	Key Features	Complexity
LangChain	General-purpose agents	Extensive tool integrations, modular design	★★
LangGraph	Complex workflows	Stateful graph-based execution, cyclic reasoning	★ ★ ★
LlamaIndex	RAG-focused applications	Advanced data connectors, query engines	★★

LangChain: A Representative Framework

We will learn:

- Using chat models with LangChain
- Scawolding and reusing prompts with templates
- Building chains to intuitively connect multi-step processes
- Inspecting inputs and outputs to chains with debug mode

Download the codebase from Canvas.



ChatOpenAI wraps OpenAI LLMs such as GPT.
With a few lines of code, you can ask questions.

```
from langchain_openai import ChatOpenAI
```

```
#load API key
```

```
...
```

```
api_key = load_api_key("api_key.txt")
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4",
```

```
    api_key=api_key,
```

```
    base_url="https://aigc-api.hkust-gz.edu.cn/v1/",
```

```
    temperature=0.7,
```

```
)
```

```
response = llm.invoke("Please classify the sentiment: I really enjoyed this course.")
```

Only for LLM Router Service, e.g., our university's GPT API

Prompt Templates

- Prompt templates represent a structured approach for dynamic prompts
- They help you build consistent, reusable prompts

```
from langchain.prompts import PromptTemplate

template_str = """You're an expert on {topic}. ...

{context}

{question}"""

prompt = PromptTemplate.from_template(template_str)
filled = prompt.format(
    topic="User feedback", context="Example review", question="Any positives?"
)
```

Assembling Labeled Instructions

- A Prompt can consist of multiple messages
- Messages can have roles, such as:
 - SystemMessage : Instructions for how the model should behave
 - HumanMessage : User questions or requests
 - AIMessage : Model responses
- ChatPromptTemplate combines multiple message prompts.

```
review_system_prompt = SystemMessagePromptTemplate(  
    prompt=PromptTemplate(  
        input_variables=["context"], template=review_system_template_str  
    )  
)  
review_human_prompt = HumanMessagePromptTemplate(  
    prompt=PromptTemplate(input_variables=["question"], template="{question}")  
)  
messages = [review_system_prompt, review_human_prompt]  
review_prompt_template = ChatPromptTemplate(  
    input_variables=["context", "question"],  
    messages=messages,  
)
```

Chains connect multiple runnable components:

- Prompt templates
- Chat models
- Output parsers
- ... and more!

Create a chain with the **|** operator

```
review_chain = review_prompt_template | chat_model
```

One call to `.invoke()` runs every step in sequence, passing outputs of a previous step as inputs to the next step.

```
review_chain = review_prompt_template | chat_model  
review_chain.invoke(<inputs>)
```

Code Check: run `chain.py`, compare with `template.py`

You can add runnable components to your chain:

```
from langchain_core.output_parsers import StrOutputParser

# ...

output_parser = StrOutputParser()

review_chain_ext = review_prompt_template | chat_model | output_parser

review_chain_ext.invoke({"context": context, "question": question})
```

Inspect Your Chain

What happened? Get detailed logging output with debug mode:

```
from langchain_core.globals import set_debug
set_debug(True)
```

```
[chain/start] [chain:RunnableSequence] Entering Chain run with input:
{
  "context": "I had a great stay!",
  "question": "Did anyone have a positive experience?"
}
[chain/start] [chain:RunnableSequence > prompt:ChatPromptTemplate] Entering Prompt run with input:
{
  "context": "I had a great stay!",
  "question": "Did anyone have a positive experience?"
}
[chain/end] [chain:RunnableSequence > prompt:ChatPromptTemplate] [0ms] Exiting Prompt run with output:
[outputs]
[llm/start] [chain:RunnableSequence > llm:ChatOpenAI] Entering LLM run with input:
{
  "prompts": [
    "System: Your job is to use patient\nreviews to answer questions about their experience at a\nhospital. Use the following context to a"
  ]
}
[llm/end] [chain:RunnableSequence > llm:ChatOpenAI] [1.17s] Exiting LLM run with output:
```

In the `langchain_proj` directory, the `reviews.py` file contains a dictionary of reviews.

Feed these reviews into the LangChain chain we built previously, and use the LLM to extract meaningful insights from the data.

■ Think: How to deal with millions of records?

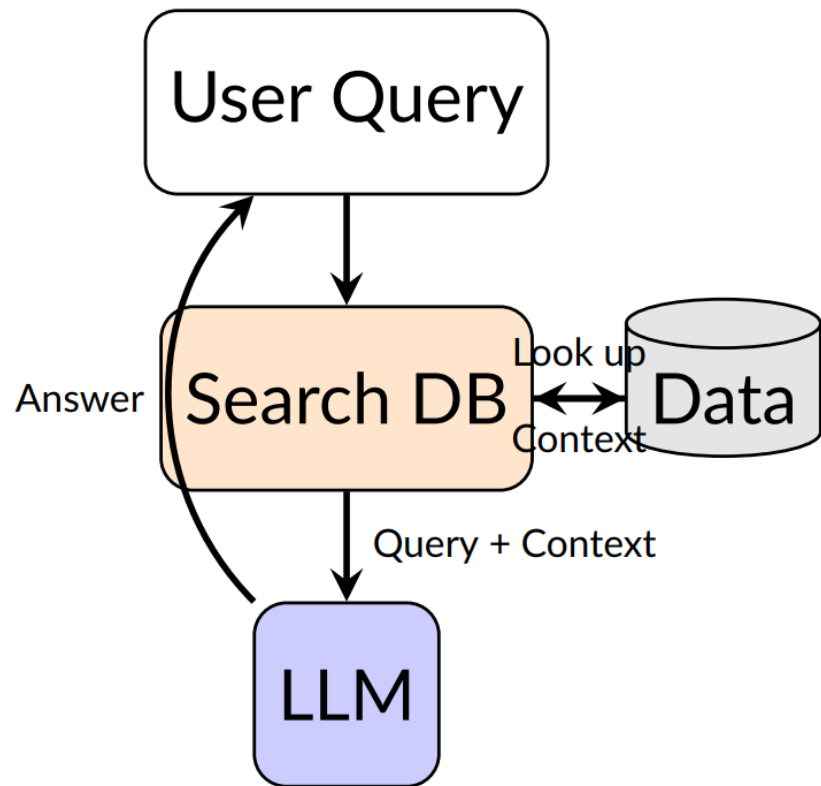
Problem:

- So far, we manually passed reviews in as context for the question.
- While this can work for a small number of reviews, it does not scale well.
- Even if we can fit all reviews into the model's context window, there's no guarantee it will use the correct reviews when answering a question.

Solution: retrieval-augmented generation (RAG)

External Memory of AI Agent: RAG

RAG: retrieving relevant documents and passing them to a language model to answer questions



Think of RAG as giving the Agent an Open Book Exam:

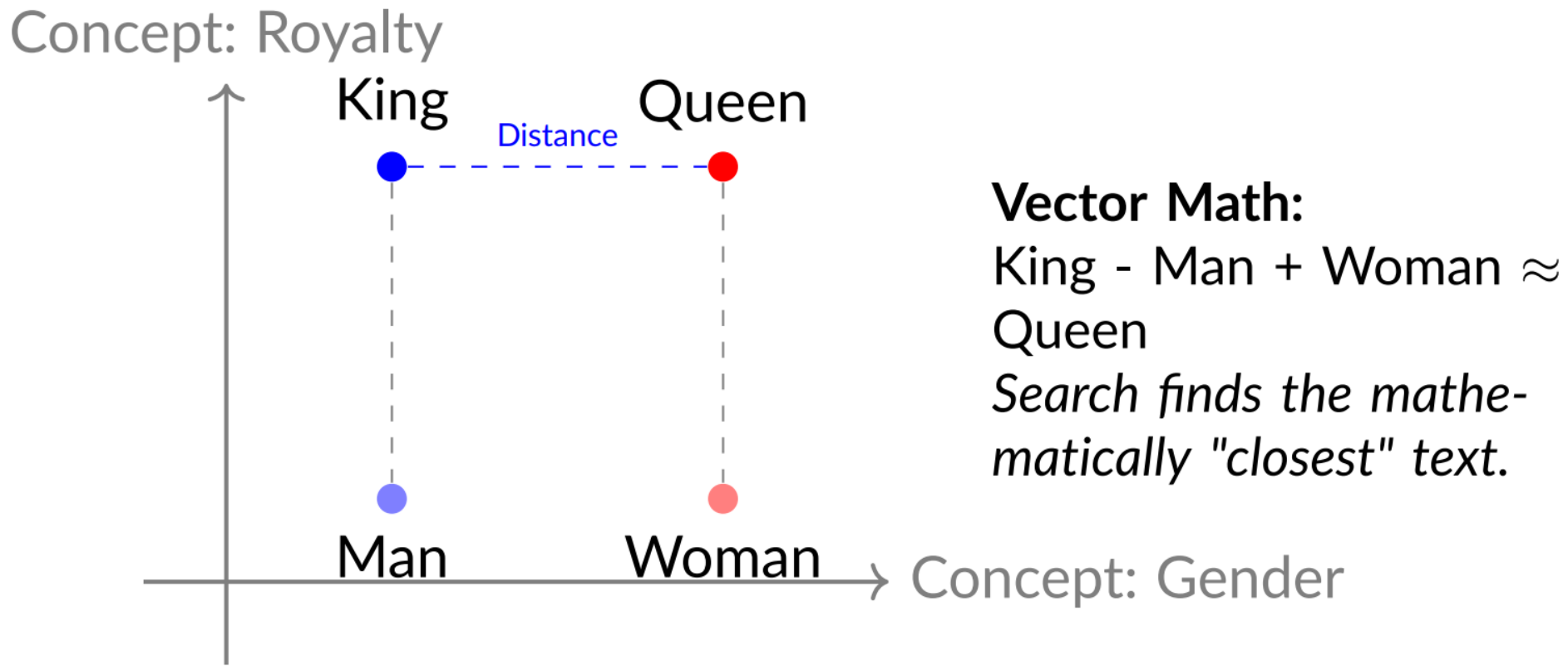
Question: User asks about private data.

Retrieval: System searches the "Book" (Database) for relevant pages.

Generation: System gives the specific pages + question to the LLM.

How Retrieval Works: Embeddings

How does a computer know that "King" is related to "Queen"?
Embeddings turn text into lists of numbers (Vectors).



Vector Similarity

Vector norm (magnitude)

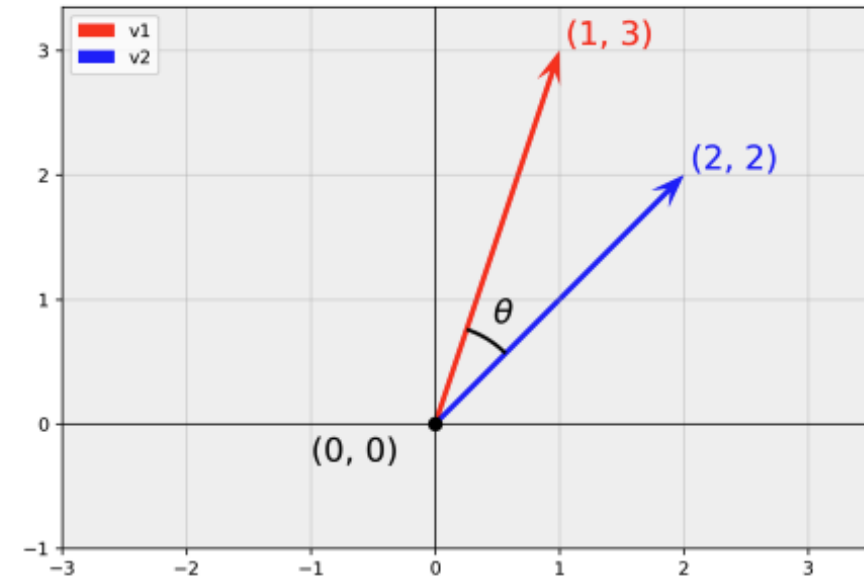
- $\|a\| = \sqrt{\sum a_i^2}$

Dot product of two vectors

- $a \cdot b = \|a\| \|b\| \cos(\theta) = \sum a_i b_i$

Cosine similarity

- $S_c(a, b) = \cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$



```
>>> import math
>>> a = [1, 3]
>>> b = [2, 2]
>>> a_dot_b = (a[0] * b[0]) + (a[1] * b[1])
>>> a_norm = math.sqrt(a[0]**2 + a[1]**2)
>>> b_norm = math.sqrt(b[0]**2 + b[1]**2)
>>> a_dot_b / (a_norm * b_norm)
0.8944271909999159
```

Vector Similarity

Vector norm (magnitude)

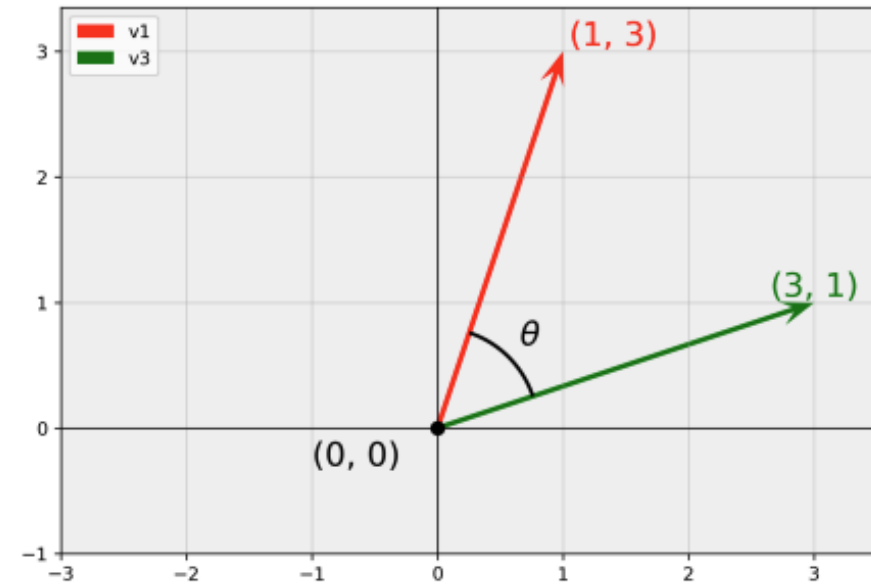
- $\|a\| = \sqrt{\sum a_i^2}$

Dot product of two vectors

- $a \cdot b = \|a\| \|b\| \cos(\theta) = \sum a_i b_i$

Cosine similarity

- $S_c(a, b) = \cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$



```
>>> import math
>>> a = [1, 3]
>>> b = [3, 1]
>>> a_dot_b = (a[0] * b[0]) + (a[1] * b[1])
>>> a_norm = math.sqrt(a[0]**2 + a[1]**2)
>>> b_norm = math.sqrt(b[0]**2 + b[1]**2)
>>> a_dot_b / (a_norm * b_norm)
0.5999999999999999
```

How to obtain Text Embeddings

- The most efficient way to generate text embeddings is to use pretrained models.
- These models vary in size, but they're all typically trained on a large corpus of text, enabling them to pick up on complex semantic relationships.
- Python Library: **SentenceTransformers**

```
(venv) $ python -m pip install sentence-transformers
```

SentenceTransformers Usage

- Different pretrained models can be swapped in easily, depending on needs such as speed, accuracy, or domain specificity.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
texts = [
    "The canine barked loudly.",
    "The dog made a noisy bark.",
    "He ate a lot of pizza.",
    "He devoured a large quantity of pizza pie.", ]
text_embeddings = model.encode(texts)

type(text_embeddings) # <class 'numpy.ndarray'>
text_embeddings.shape # (4, 384)
```

■ Compare them using cosine similarity

```
import numpy as np

def compute_cosine_similarity(u: np.ndarray, v: np.ndarray) -> float:
    """Compute the cosine similarity between two vectors"""
    return (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))

print(compute_cosine_similarity(text_embeddings[0,],text_embeddings[1,]))
print(compute_cosine_similarity(text_embeddings[1,],text_embeddings[2,]))
print(compute_cosine_similarity(text_embeddings[2,],text_embeddings[3,]))
print(compute_cosine_similarity(text_embeddings[3,],text_embeddings[0,]))
```

A vector database is a database that allows you to efficiently store and query embedding data.

- They store embeddings representing text, images, or other unstructured data.
- They allow **fast similarity search** across large datasets
- Instead of exact matches, they **find closest vectors using similarity metrics**.
- Enable applications like semantic search, recommendation systems, and **providing relevant context to large language models**

Components of Vector Database

Embedding function: we often store and query data in its raw form. Internally, the vector database needs to know how to convert your data to embeddings, and you have to specify an embedding function for this.

Similarity metric: To assess embedding similarity, you need a similarity metric like cosine similarity, the dot product, or Euclidean distance.

Indexing: When you're dealing with a large number of embeddings, comparing a query embedding to every embedding stored in the database is often too slow. To overcome this, vector databases employ indexing algorithms that group similar embeddings together.

Metadata: You can store metadata with each embedding to help give context and make query results more precise, e.g., filter your embedding searches on metadata.

CRUD operations: Most vector databases support create, read, update, and delete (CRUD) operations.

ChromaDB: A Representative Vector Database

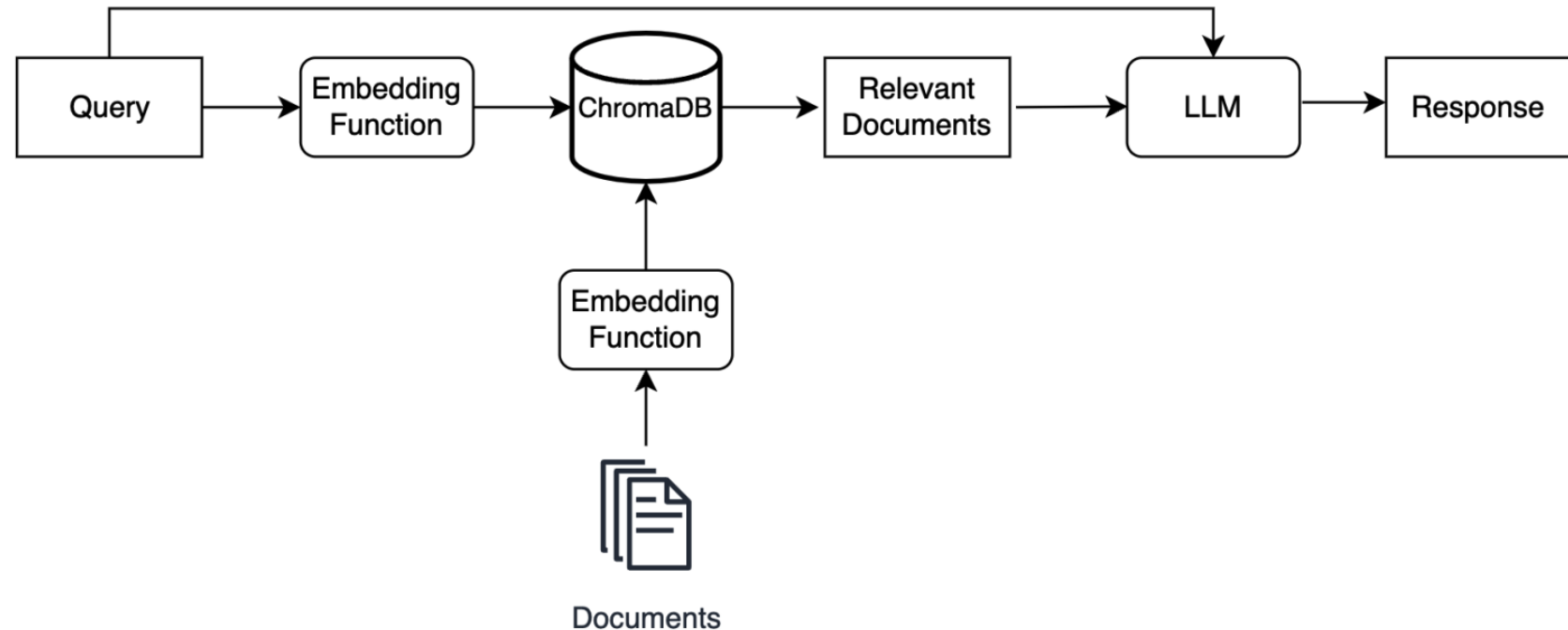
- ChromaDB is an open-source vector database made to work with LLMs
- Provides a developer-friendly API for working with embeddings
- Optimized for fast similarity search and semantic retrieval
- Compatible with Langchain

```
(venv) $ python -m pip install chromadb  
(venv) $ python -m pip install langchain_chroma
```

<https://docs.trychroma.com/docs/overview/getting-started>

RAG Pipeline: From Document to Answer

- Ingestion (Offline): Break documents into small "Chunks" and save them as vectors.
- Retrieval (Runtime): Find chunks similar to the question.
- Synthesis: Feed chunks to LLM



ChromaDB Usage in Langchain

Create a ChromaDB vector database with patient reviews:

```
from langchain_community.document_loaders import CSVLoader
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

REVIEWS_CSV_PATH = "data/reviews.csv"
loader = CSVLoader(file_path=REVIEWS_CSV_PATH, source_column="review")
reviews = loader.load()

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={"device": "cpu"},
    encode_kwargs={"normalize_embeddings": True},
)

REVIEWS_CHROMA_PATH = "chroma_data"
reviews_vector_db = Chroma.from_documents( documents=reviews,
embedding=embedding_model, persist_directory=REVIEWS_CHROMA_PATH,)
```

ChromaDB Usage in Langchain

- In practice, if you're embedding a large document, you should use a text splitter.
- Text splitters break the document into smaller chunks before running them through an embedding model.
- Embedding models have a **fixed-size context window**, and as the size of the text grows, an embedding's ability to accurately represent the text decreases.

```
(venv) $ python create_retriever.py
```

Performing semantic search over the review embeddings

```
question = """Has anyone complained about communication with the hospital staff?"""
```

```
relevant_docs = reviews_vector_db.similarity_search(question, k=3)
```

```
relevant_docs[0].page_content  
relevant_docs[1].page_content  
relevant_docs[2].page_content
```

This creates an embedding for the question and searches the vector database for the three most similar review embeddings to question embedding.

ChromaDB Usage in Langchain

Add reviews retriever to review_chain so that relevant reviews are passed to the prompt as context.

```
from langchain_core.runnables import RunnablePassthrough

reviews_retriever = reviews_vector_db.as_retriever(k=3)

review_chain = (
    {"context": reviews_retriever, "question": RunnablePassthrough()}
    | review_prompt_template
    | chat_model
    | StrOutputParser()
)

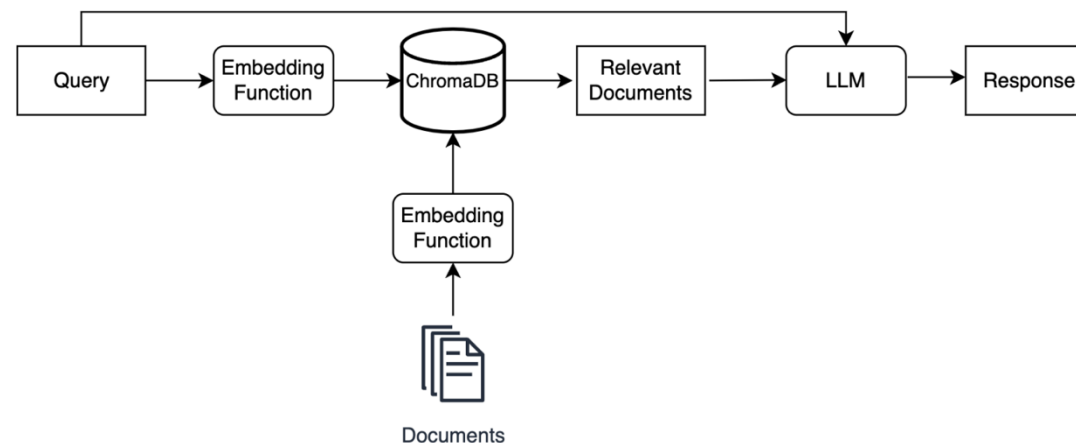
question = """Has anyone complained about communication with the hospital staff?"""
response = review_chain.invoke(question)
print(response)
```

Assigning question to a RunnablePassthrough object ensures the question gets passed unchanged to the next step in the chain.

Based on the current code:

1. Change the embedding model: [sentence-transformers/all-mpnet-base-v2](#), rebuild the Chroma vector database.
2. Change the retriever to return top-10 documents.
3. Before and after each change, compare:
 - Retrieval relevance
 - Answer quality
 - Inference speed

- We created a chain to answer questions using patient reviews.
- What if we want our chatbot to also answer questions about other hospital data, such as hospital wait times?
- Advanced systems do not just search blindly.
- They use an Agent to seamlessly switch between answering patient review and wait time questions depending on the user's query.



Component:

- The patient review chain
- A function that can look up wait times at a hospital
- A way for an LLM to know when it should answer questions about patient experiences or look up wait times → **Agent**
- Chains: the sequence of actions is hard-coded
- Agents: use a language model to determine which actions to take and in which order.

First, we create a tool

```
import random
import time

def get_current_wait_time(hospital: str) -> int | str:
    """Dummy function to generate fake wait times"""

    if hospital not in ["A", "B", "C", "D"]:
        return f"Hospital {hospital} does not exist"

    # Simulate API call delay
    time.sleep(1)

    return random.randint(0, 10000)
```

Then, define a Tool interface that an agent uses to interact with a function.

```
from langchain_core.tools import Tool

tools = [
    Tool(
        name="Reviews",
        func=review_chain.invoke,
        description="""Useful when you need to answer questions about patient reviews or experiences at the hospital.....""",
    ),
    Tool(
        name="Waits",
        func=get_current_wait_time,
        description="""Use when asked about current wait times at a specific hospital. ....""",
    ),
]
```

This creates an agent designed to pass inputs to functions.

```
from langchain.agents import create_agent

agent_chat_model = ChatOpenAI(
    model="gpt-4",
    api_key=api_key,
    base_url="https://aigc-api.hkust-gz.edu.cn/v1/",
    temperature=0.2,
)

hospital_agent = create_agent(
    model=agent_chat_model,
    tools=tools,
    system_prompt="You are a helpful assistant. Use tools when appropriate.",
)
```

Unfortunately, our university's API does not support tool calling yet.

Advanced Prompting & Reasoning

- zero-shot, few-shot, delimiters, describe the request in numbered Steps
- Chain-of-Thought (CoT) Prompting, Output Formatting
- Role in Conversation
- Self-Consistency (SC)

AI Agent

- Agent Development Frameworks: LangChain
- External Memory: RAG
- Text Embeddings: SentenceTransformers
- Vector Database: ChromaDB
- Agent & Tool Use

Python Programming with Artificial Intelligence

Thank you for listening!

